

Back-End Development Fellow (Node.js / Express / MongoDB) Technical Assessment

Goal

Validate ability to build **REST APIs**, use **MongoDB Atlas + Compass**, implement auth integration, basic CI, and clean backend structure.

Timebox

- Take-home: **6–10 hours**
- Live/time-boxed: 3 Days

Project: “T-World Mini API Learning + Profiles”

Build a backend API supporting a mobile client.

Required Features

1. **Node.js + Express API**
 - TypeScript preferred (optional for Fellow; bonus)
 - Clear folder structure (`routes/`, `controllers/`, `services/`, `db/`)
2. **MongoDB (Atlas)**
 - Collections: `users`, `items`, `saved_items`
 - Index at least one field meaningfully (e.g., `items.category`, `users.email`)
3. **Endpoints**

- `POST /auth/register` (accepts email/password; can mock Firebase verification or just store)
- `POST /auth/login` (simple JWT is ok; if Firebase integration used, explain)
- `GET /items` (pagination + filter by category)
- `GET /items/:id`
- `POST /items/:id/save`
- `DELETE /items/:id/save`
- `GET /me/saved`

4. Validation + Error Handling

- Input validation (lightweight is fine)
- Centralized error handler
- Proper status codes

5. GitHub

- Repo with commits
- Basic GitHub Actions: run lint/test or at least `npm run build`

Stretch (Optional)

- Seed script for sample items
- Rate limit middleware
- Unit tests for one route/service

Deliverables

- GitHub repo

- Postman collection or curl examples
- README:
 - How to run locally
 - Env vars (`.env.example`)
 - Database schema summary

Evaluation Rubric (100 pts)

- API design & correctness (20)
- MongoDB schema choices + indexing (20)
- Auth approach & security basics (15)
- Validation + error handling (15)
- Code structure & readability (15)
- CI workflow + repo hygiene (10)
- Documentation quality (5)

Pass threshold (Fellow): 70+ with working endpoints and DB persistence.